

Catatan Praktikum Minggu 4

Mata Kuliah: Pemrograman Web

Nama: Nova Reskianti

NIM: 10241058

READ: Siklus Form Gagal dan Alur Validasi

Analisis penelusuran satu siklus form gagal (misalnya saat mengirimkan nilai SKS = 99 pada form tambah mata kuliah):

1. Method dan Controller Penerima Request

- **Controller:** `App\Http\Controllers\CourseController`
- **Method:** `store(StoreCourseRequest $request)` (atau method `update(UpdateCourseRequest $request, Course $course)` saat perubahan data).

2. Titik Terjadinya Validasi

Validasi terjadi **sebelum baris pertama kode di dalam method controller dieksekusi**.

- **Mekanisme Teknis:** Laravel memanfaatkan *Dependency Injection* melalui Service Container. Ketika container me-resolve type-hint `StoreCourseRequest`, method `validateResolved()` otomatis dieksekusi. Jika ada aturan pada `rules()` yang dilanggar, Form Request langsung melempar `Illuminate\Validation\ValidationException`. Eksekusi kode dihentikan seketika sehingga baris pertama di dalam method `store()` tidak pernah dijalankan.

3. Arah Redirect Setelah Gagal dan Penentunya

- **Tujuan Redirect:** Laravel me-redirect kembali ke URL sebelumnya (*previous URL / back*), yaitu halaman form input (`/courses/create`).
- **Penentu Tujuan:** Ditentukan oleh method `getRedirectUrl()` pada Form Request yang secara default memanggil `url()->previous()`. Nilai URL sebelumnya ini dicatat dan dikelola oleh middleware `StartSession` melalui session internal `_previous.url` atau dibaca dari HTTP header `Referer`.

4. Asal Pesan Error pada `@error('sks')`

- Pesan error diambil dari objek **View Error Bag** (`$errors`), yaitu instance dari `Illuminate\Support\ViewErrorBag` yang disuntikkan secara global ke seluruh view oleh middleware `ShareErrorsFromSession`.
- Saat `ValidationException` terpicu, Laravel menyimpan daftar pesan kesalahan ke dalam *flash session* dengan key `errors`. Direktif Blade `@error('sks')` memeriksa apakah `$errors->has('sks')` bernilai true, lalu menyediakan variabel `$message` yang berisi pesan kesalahan pertama (`$errors->first('sks')`).

5. Asal Nilai pada `old('sks')` dan Masa Berlakunya

- Nilai diambil dari *flash session* dengan key `_old_input` yang disimpan secara otomatis oleh Laravel saat melakukan redirect kembali (`withInput()`).
- **Masa Berlaku:** Hanya bertahan selama **1 siklus request berikutnya (flash session)**. Begitu halaman form selesai dirender ulang dan siklus request selesai, data `_old_input` langsung dihapus secara otomatis dari session.

6. Nama Cookie Session Laravel

- Pada peramban (*DevTools* → *Application* → *Cookies*), nama cookie session Laravel default adalah: `kampuslms_session` (dihasilkan dari `Str::slug(env('APP_NAME', 'laravel')) . '_session'`).
- Selain itu, terdapat cookie pendamping yaitu `XSRF-TOKEN` yang berisi token CSRF terenkripsi untuk keamanan request AJAX/JavaScript.

BREAK: Tujuh Kerusakan (Rusak dengan Sengaja)

Eksperimen merusak sistem secara sengaja untuk mengamati mekanisme pertahanan Laravel:

#	Yang Dirusak	Prediksi Sebelum Mencoba	Hasil/Error Sebenarnya
1	Hapus <code>@csrf</code> dari form, lalu kirim	Request POST akan ditolak server karena tidak memiliki token autentikasi form.	Muncul halaman error HTTP 419 Page Expired . Middleware <code>ValidateCsrfToken</code> memblokir request karena ketiadaan token <code>_token</code> .
2	Ganti <code>\$request->validated()</code> menjadi <code>\$request->all()</code> , lalu kirim field liar lewat <code>curl</code>	Field ekstra yang disusupkan akan lolos dan berisiko masuk ke database.	Mass assignment terbuka kembali . Field asing/liar yang dikirim penyerang ikut diproses oleh model. Jika model mengizinkan kolom tersebut, integritas data terancam.
3	Hapus validasi <code>exists:users,id</code> pada <code>lecturer_id</code> , kirim <code>lecturer_id=99999</code>	Database akan menolak foreign key palsu atau menghasilkan data yatim (<i>orphan data</i>).	Terjadi SQL exception 500 Server Error (<code>QueryException: Integrity constraint violation: 1452 Cannot add or update a child row</code>). Jika foreign key database longgar, data yatim akan tersimpan dan memicu error fatal saat relasi dosen dipanggil.

#	Yang Dirusak	Prediksi Sebelum Mencoba	Hasil/Error Sebenarnya
4	Hapus validasi <code>in:...</code> pada <code>status</code> , kirim <code>status=superadmin</code>	Nilai status sembarangan akan masuk ke sistem karena tidak ada batasan enum.	Nilai <code>superadmin</code> tersimpan di database. Akibatnya, query filter status menjadi kacau dan tampilan badge status pada view menjadi rusak (<i>broken UI/logic</i>).
5	Hapus - <code>>withQueryString()</code> , lakukan pencarian lalu klik halaman 2	Parameter pencarian akan hilang saat berpindah ke halaman berikutnya.	Filter pencarian hilang (bug klasik) . URL berubah menjadi <code>/courses?page=2</code> tanpa menyertakan query pencarian (<code>q</code>). Daftar data kembali menampilkan semua mata kuliah tanpa filter.
6	Ganti <code>return redirect()</code> menjadi <code>return view()</code> pada <code>store</code> , lalu tekan F5 setelah simpan	Browser akan menanyakan konfirmasi pengiriman ulang dan data berisiko terduplikasi.	Muncul dialog browser " Confirm Form Resubmission ". Jika user mengonfirmasi, request POST terkirim ulang dan data mata kuliah yang sama tersimpan dua kali (duplikasi data) . URL di address bar tetap tertinggal di <code>/courses</code> (POST).
7	Hapus <code>old(...)</code> dari semua input, lalu kirim form dengan satu kesalahan	Isian form akan hilang seluruhnya ketika validasi gagal.	Seluruh kolom input kembali kosong. Pengguna terpaksa mengetik ulang seluruh formulir dari awal hanya karena salah mengisi satu field, merusak kenyamanan pengguna (<i>bad UX</i>).

Pengujian Mass Assignment & Input Liar via cURL

Pengujian dilakukan menggunakan perintah terminal cURL untuk membuktikan bahwa validasi frontend sama sekali tidak melindungi endpoint server:

```
curl -X POST http://kampuslms.test/courses \
-H "X-CSRF-TOKEN: <token_csrf_aktif>" \
-b cookies.txt \
-d "code=XX01" \
-d "name=Uji Penetrasi" \
-d "sks=3" \
-d "lecturer_id=99999" \
-d "status=superadmin" \
-d "is_admin=1"
```

- **Hasil Pengamatan:** Jika server mengandalkan `$request->all()` dan tidak memiliki aturan `exists` atau `in`, seluruh data liar di atas akan diproses. Penggunaan `$request->validated()` bersama `FormRequest` terbukti menjadi benteng utama pertahanan server.

FIX: Perbaiki Repo Cacat (6 Masalah)

Analisis dan perbaikan 6 masalah pada modul mata kuliah branch `w04`:

1. Validasi Hanya di Frontend

- **Perbaikan:** Menghapus ketergantungan validasi pada skrip browser semata, lalu membuat kelas Form Request tersendiri (`StoreCourseRequest` dan `UpdateCourseRequest`) dengan aturan ketat di server-side (`required`, `max`, `exists`, `in`, `between`).
- **Dampak Pengguna:** Memberikan pesan validasi yang akurat dan konsisten meskipun pengguna menggunakan browser versi lama atau menonaktifkan fitur JavaScript.
- **Dampak Penyerang:** Menutup celah bypass. Penyerang yang menembak API/endpoint langsung menggunakan cURL, Postman, atau manipulasi DOM langsung dihentikan dengan status respons **HTTP 422 Unprocessable Content**.

2. Aturan `unique` pada Update yang Menolak Dirinya Sendiri

- **Perbaikan:** Menambahkan method `->ignore($this->course)` pada aturan keunikan di `UpdateCourseRequest`:

```
Rule::unique('courses', 'code')->ignore($this->course)
```

- **Dampak Pengguna:** Pengguna dapat memperbarui nama, SKS, atau dosen mata kuliah tanpa terhalang pesan error palsu bahwa kode mata kuliah sudah digunakan.
- **Dampak Penyerang:** Penyerang tetap dicegah menduplikasi kode mata kuliah milik mata kuliah lain, menjaga integritas keunikan data di database.

3. Filter Disimpan di Session

- **Perbaikan:** Mengubah logika filter dari Session ke Query String menggunakan `$request->filled('q')` dan `$request->filled('status')` di controller:

```
$courses = Course::query()
    ->when($request->filled('q'), fn($q) => $q->where('name', 'like', "%{$request->q}%"))
    ->when($request->filled('status'), fn($q) => $q->where('status',
$request->status))
    ->paginate(15)
    ->withQueryString();
```

- **Dampak Pengguna:** URL pencarian menjadi *shareable* (bisa dibagikan ke rekan lain), mendukung bookmark, tombol *Back/Forward* browser bekerja normal, dan tidak terjadi konflik data saat membuka aplikasi di banyak tab sekaligus.
- **Dampak Penyerang:** Mencegah kebocoran memori session (*session bloat*) serta manipulasi state pengguna yang tidak diinginkan.

4. Pagination Kehilangan Query String

- **Perbaikan:** Menyertakan method chaining `->withQueryString()` pada pemanggilan pagination di `CourseController::index()`.
- **Dampak Pengguna:** Pengguna dapat berpindah ke halaman 2, 3, dan seterusnya tanpa kehilangan kata kunci pencarian atau filter status yang sedang aktif.
- **Dampak Penyerang:** Memastikan parameter filter tetap konsisten dan terikat secara eksplisit pada setiap request pagination.

5. `store` Tanpa Redirect (Mengembalikan View Langsung)

- **Perbaikan:** Menerapkan pola **PRG (Post/Redirect/Get)** secara konsisten. Mengganti `return view(...)` menjadi:

```
return redirect()->route('courses.index')->with('success', 'Mata kuliah
berhasil ditambahkan.');
```

- **Dampak Pengguna:** Pengguna terbebas dari dialog peringatan "*Confirm Form Resubmission*" saat me-refresh browser, dan navigasi alamat URL tetap bersih dan benar.
- **Dampak Penyerang:** Mencegah terjadinya pengiriman ulang data secara tidak sengaja maupun serangan spam form duplikasi data berbasis refresh halaman.

6. Form Tanpa `@csrf`

- **Perbaikan:** Menyisipkan direktif Blade `@csrf` di dalam seluruh tag `<form method="POST">`:

```
<form method="POST" action="{{ route('courses.store') }}">
    @csrf
    ...
</form>
```

- **Dampak Pengguna:** Formulir dapat dikirim dengan sukses tanpa menemui pesan kegagalan sesi **HTTP 419 Page Expired**.
- **Dampak Penyerang:** Mencegah serangan **Cross-Site Request Forgery (CSRF)** di mana situs jahat pihak ketiga mencoba memanfaatkan sesi login korban untuk memanipulasi atau menghapus data mata kuliah.

BUILD: Form dan Daftar yang Layak Pakai

Implementasi fitur formulir dan daftar data yang layak pakai, teruji, dan aman pada aplikasi KampusLMS:

1. `StoreCourseRequest` dan `UpdateCourseRequest` Lengkap dengan Pesan Bahasa Indonesia

Membuat kelas Form Request terpisah di direktori `app/Http/Requests/`:

- **`StoreCourseRequest.php`:**
 - Menetapkan aturan validasi: `code` (unik di tabel `courses`), `name` (maksimal 150 karakter), `sks` (integer antara 1–6), `lecturer_id` (wajib ada di tabel `users`), dan `status` (`in:draft,active,archived`).
 - Menyediakan array `messages()` dalam Bahasa Indonesia yang informatif dan ramah pengguna.
- **`UpdateCourseRequest.php`:**
 - Menerapkan aturan keunikan dengan pengecualian ID record saat pembaruan data:

```
Rule::unique('courses', 'code')->ignore($this->course)
```

- Menjaga konsistensi aturan data lain dan pesan kesalahan berbahasa Indonesia.

2. Formulir Tambah dan Edit Mata Kuliah yang Ramah Pengguna

Implementasi pada `resources/views/courses/create.blade.php` dan `courses/edit.blade.php`:

- **Proteksi `@csrf` dan Spoofing Method `@method('PUT')`**: Menjamin keamanan request dari serangan pemalsuan origin.
- **Retensi Input dengan `old()`**:
 - Pada form tambah: `value="{{ old('code') }}"` dan `value="{{ old('sks', 3) }}"`.
 - Pada form edit: `value="{{ old('code', $course->code) }}"` dan `value="{{ old('sks', $course->sks) }}"`.
- **Umpan Balik Kesalahan (*Error Feedback*)**:
 - Menampilkan pesan kesalahan spesifik tepat di bawah tiap input dengan `@error('field')` ... `@enderror`.
 - Menampilkan kotak peringatan ringkasan kesalahan di bagian atas form menggunakan `@if($errors->any())`.

3. Flash Message Sukses dan Gagal Global di Layout

Implementasi pada `resources/views/components/layout.blade.php`:

- Ditempatkan di luar slot konten utama sehingga notifikasi otomatis aktif dan konsisten di seluruh halaman aplikasi.
- Menggunakan blok kondisional:

```
@if (session('success'))
    <div id="flash-success" role="status" aria-live="polite" class="...">
```

```

        <span>{{ session('success') }}</span>
        <button type="button" onclick="dismissFlash('flash-
success')">X</button>
    </div>
@endif

@if (session('error'))
    <div id="flash-error" role="alert" aria-live="assertive" class="...">
        <span>{{ session('error') }}</span>
        <button type="button" onclick="dismissFlash('flash-
error')">X</button>
    </div>
@endif

```

- Dilengkapi dengan styling utilitarian modern, tombol penutup (*dismiss*), serta skrip auto-dismiss setelah beberapa detik.

4. Daftar Mata Kuliah dengan Pencarian, Filter Status, dan Pagination 15 per Halaman

Implementasi pada `CourseController::index()` dan `resources/views/courses/index.blade.php`:

- **Controller Query Builder:**

```

$courses = Course::query()
    ->with('lecturer') // Eager loading mencegah N+1 query
    ->when($request->filled('q'), function ($query) use ($request) {
        $query->where(function ($q) use ($request) {
            $q->where('name', 'like', '%' . $request->q . '%')
            ->orWhere('code', 'like', '%' . $request->q . '%');
        });
    })
    ->when($request->filled('status'), fn ($query) => $query-
>where('status', $request->status))
    ->latest()
    ->paginate(15)
    ->withQueryString(); // Mempertahankan state filter di pagination

```

- **Form Filter & Pencarian:** Menggunakan metode GET yang sinkron dengan query string URL, dilengkapi tombol filter status yang mempertahankan parameter pencarian.
- **Pagination View:** Memanggil `{{ $courses->links() }}` di bawah tabel/grid mata kuliah.

5. Penerapan pada Modul Pengguna (User Management)

Menerapkan arsitektur dan standar keamanan yang sama pada modul pengguna:

- **Form Request Pengguna:**
 - `StoreUserRequest`: Memvalidasi `name`, `email` (`unique:users,email`), `password` (minimal 8 karakter), `role` (`in:admin,dosen,mahasiswa`), dan `nim_nip` (`unique:users,nim_nip`).

- `UpdateUserRequest`: Menggunakan `Rule::unique('users', 'email')->ignore($this->user)` dan `Rule::unique('users', 'nim_nip')->ignore($this->user)`.
- **UserController.php**:
 - Menerapkan pencarian berdasarkan nama, email, atau NIM/NIP, filter berdasarkan role pengguna, serta pagination 15 per halaman dengan chaining `->withQueryString()`.
 - Pengisian field sensitif **role** dilakukan secara eksplisit pada controller untuk mencegah kebocoran mass assignment.
- **Tampilan Formulir Pengguna**: Dilengkapi dengan `@csrf`, retensi input `old()`, pesan `@error`, dan `@method('PUT')`.

6. Konfirmasi Sebelum Menghapus dan Penggunaan Method DELETE

- Seluruh operasi penghapusan data tidak menggunakan tautan biasa (`<a> GET`), melainkan formulir mandiri dengan spoofing method HTTP **DELETE** dan proteksi `@csrf`.
- Dilengkapi dengan dialog konfirmasi (*confirmation prompt*) berbasis JavaScript untuk mencegah penghapusan data secara tidak sengaja oleh pengguna:

```
<form action="{{ route('courses.destroy', $course) }}" method="POST"
onsubmit="return confirm('Apakah Anda yakin ingin menghapus mata kuliah
{{ $course->name }}?')" class="inline">
  @csrf
  @method('DELETE')
  <button type="submit" class="btn-danger-soft">
    Hapus
  </button>
</form>
```

- Penerapan yang sama berlaku pada modul Pengguna (`route('users.destroy', $user)`).

Checkpoint Minggu 4

1. Kenapa validasi di JavaScript tidak dianggap keamanan? Peragakan cara melewatinya.

- **Kenapa bukan keamanan?**
 Karena JavaScript berjalan di browser pengguna (laptop atau HP), bukan di server kita. Apa pun yang ada di browser pengguna bisa dimatikan, diedit, atau dilewati dengan sangat mudah. Validasi JavaScript itu fungsinya cuma **pemberitahuan ramah untuk pengguna** agar tahu kalau ada salah ketik tanpa harus menunggu loading server. Validasi JavaScript **bukan satpam keamanan**. Keamanan yang sesungguhnya wajib ada di server (seperti `StoreCourseRequest` di Laravel).
- **Cara melewatinya (sangat gampang):**
 1. **Matikan JavaScript**: Buka menu pengaturan di Developer Tools browser, centang *Disable JavaScript*. Form bisa langsung dikirim tanpa ada pemeriksaan sama sekali.

2. **Lewat Console (F12):** Buka form tambah data, tekan tombol F12, buka tab **Console**, lalu ketik:

```
document.querySelector('form').submit();
```

Form akan langsung terkirim dan mengabaikan semua aturan **required** maupun batasan panjang teks.

3. **Tembak langsung tanpa buka browser (cURL / Postman):** Penyerang bisa langsung mengirim data kosong atau SKS ngawur (**sks=999**) langsung ke server lewat terminal:

```
curl -X POST http://localhost:8000/courses -d  
"code=&name=&sks=999&lecturer_id=99999" ...
```

Jika server tidak memvalidasi, data rusak tersebut akan langsung masuk ke database.

2. Apa yang dikembalikan `$request->validated()` dan kenapa lebih aman daripada `$request->all()`?

- **Bedanya:**

- `$request->all()` mengambil **semua data apa adanya** yang dikirim dari form, termasuk data selundupan yang sengaja ditambahkan penyerang.
- `$request->validated()` hanya mengambil **data yang memang terdaftar di aturan validasi dan terbukti lolos seleksi**.

- **Kenapa lebih aman?**

Untuk mencegah bahaya **Mass Assignment** (penyelundupan data yang tidak diinginkan).

- **Contoh sederhana di KampusLMS:**

Bayangkan di form tambah pengguna, penyerang iseng menyelipkan input tambahan **role=admin**.

- Kalau kita pakai `$request->all()`, data **role=admin** bisa ikut tersimpan ke database, sehingga pengguna biasa tiba-tiba berubah jadi admin.
- Kalau kita pakai `$request->validated()`, Laravel akan menyaring data dengan ketat. Hanya field resmi yang diambil, sedangkan data selundupan **role=admin** otomatis langsung dibuang.

3. Jelaskan pola PRG. Apa yang terjadi kalau store mengembalikan view?

- **Apa itu pola PRG (Post $\rightarrow$ Redirect $\rightarrow$ Get)?**

PRG adalah aturan kebiasaan yang benar saat membuat formulir:

1. **POST:** Pengguna menekan tombol simpan form $\rightarrow$ browser mengirim data ke server.

2. **REDIRECT:** Server menyimpan data ke database, lalu **menyuruh browser pindah halaman** (`redirect()->route('courses.index')`).
3. **GET:** Browser membuka halaman tujuan baru tersebut dengan santai lewat permintaan biasa (GET).

- **Apa akibatnya kalau setelah menyimpan, controller langsung memanggil `return view(...)`?**

1. **Data tersimpan dobel saat refresh (F5):** Browser masih menganggap sedang berada di proses kirim form. Jika pengguna menekan tombol F5 atau refresh, browser akan memunculkan peringatan "*Confirm Form Resubmission*". Jika pengguna klik lanjut, data mata kuliah yang sama akan tersimpan dua kali ke database.
2. **Alamat URL jadi salah:** Alamat di browser tetap tertahan di URL penyimpanan (`/courses`), bukan halaman semestinya, sehingga pengguna tidak bisa mem-bookmark halaman tersebut dan tombol Back/Forward browser menjadi kacau.

4. Kenapa filter pencarian sebaiknya di query string, bukan session? Beri satu skenario yang rusak kalau dipindah ke session.

- **Kenapa harus di query string (tanda tanya di URL, contoh: `?q=web&status=active`):**

1. **Bisa dicopy dan dibagikan:** Tautan pencarian bisa kita kirim lewat chat ke teman atau dosen, dan saat mereka buka, hasilnya sama persis.
2. **Tombol Back dan Forward browser tidak bingung:** Setiap kali kita mengganti kata kunci pencarian, browser mencatatnya di riwayat halaman.
3. **Pindah halaman (Pagination) tetap aman:** Chaining `->withQueryString()` memastikan kata kunci pencarian tidak mendadak hilang saat kita klik Halaman 2 atau 3.
4. **Tiap tab browser mandiri:** Membuka banyak tab tidak akan saling mengacaukan hasil pencarian.

- **Satu skenario yang rusak kalau disimpan di Session (Masalah Dua Tab):**

1. Dosen membuka **Tab 1** mencari mata kuliah "*Pemrograman Web*". Filter ini tersimpan di Session server.
2. Lalu dosen membuka **Tab 2** mencari mata kuliah "*Kalkulus*". Filter di Session server sekarang tertimpa oleh pencarian Tab 2.
3. Dosen kembali ke **Tab 1** dan mengklik **Halaman 2**.
4. **Hasil yang rusak:** Karena server membaca filter dari Session yang sudah tertimpa, isi Tab 1 yang tadinya "*Pemrograman Web*" tiba-tiba malah menampilkan data mata kuliah "*Kalkulus*". Tampilan Tab 1 jadi rusak dan tidak sesuai.

5. Apa fungsi `@csrf`? Serangan apa yang dicegahnya, dan bagaimana serangan itu bekerja?

- **Fungsi `@csrf`:**

Membuat input rahasia yang tidak terlihat di form (`<input type="hidden" name="_token" ...>`). Isinya kode rahasia acak unik milik pengguna. Ini berfungsi seperti "**stempel resmi**" dari web

KampusLMS. Server hanya mau memproses form jika stempel rahasianya cocok dengan sesi pengguna.

- **Serangan yang dicegah:**

CSRF (Cross-Site Request Forgery) — serangan tipuan di mana website luar memanfaatkan akun korban yang sedang aktif login untuk melakukan tindakan berbahaya tanpa disadari korban.

- **Cara kerja serangannya (secara sederhana):**

1. Anda sedang login sebagai Admin di KampusLMS (sesi login masih aktif di browser).
2. Tanpa logout, Anda membuka tab lain dan mengklik link website penipu (misalnya web jebakan undian hadiah).
3. Di dalam web jebakan itu, ada tombol atau skrip otomatis yang mengirim perintah hapus mata kuliah ke KampusLMS.
4. Karena dikirim dari browser Anda, browser otomatis membawa tanda pengenal login Admin Anda.
5. **Jika tanpa @csrf:** KampusLMS percaya begitu saja karena tanda pengenal loginnya benar, sehingga mata kuliah langsung terhapus tanpa Anda sadari!
6. **Jika memakai @csrf:** Website luar tidak bisa membaca kode stempel rahasia Anda. Karena form kiriman penipu tidak punya stempel @csrf yang cocok, Laravel langsung menolaknya dengan error **HTTP 419 Page Expired**.

6. Kenapa aturan unique pada update perlu `ignore()`?

- **Masalahnya:**

- Misalnya ada mata kuliah dengan kode `SI101`.
- Suatu hari kita ingin mengedit nama dosennya saja, sedangkan kodenya tetap `SI101`.
- Saat tombol simpan diklik, sistem mengecek ke database: *"Apakah kode SI101 sudah ada yang pakai?"*.
- Database menjawab: *"Sudah ada!"* (padahal itu data milik mata kuliah itu sendiri yang sedang diedit).
- Tanpa `ignore()`, sistem mengira ada data ganda dan menolak disimpan dengan pesan error: *"Kode mata kuliah ini sudah digunakan"*.

- **Solusinya menggunakan `ignore()`:**

Method `ignore()` memberitahu Laravel: *"Tolong cek apakah kodenya kembar dengan mata kuliah lain, TAPI kecuali mata kuliah yang sedang saya edit ini!"*.

Contoh pada `UpdateCourseRequest.php`:

```
Rule::unique('courses', 'code')->ignore($this->course)
```

Dan pada `UpdateUserRequest.php`:

```
Rule::unique('users', 'email')->ignore($this->user)
```

Dengan cara ini, pengguna bisa menyimpan perubahan datanya sendiri tanpa terblokir, tetapi jika pengguna mengganti kodenya menjadi kode milik mata kuliah lain yang sudah ada, sistem tetap akan melarangnya.